Lab 7
Joins

queries.sql

```sql
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(30) UNIQUE
);
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
SELECT Student.student_id,
       Student.student_name,
       Department.dept_name
FROM Student
INNER JOIN Department
ON Student.dept_id = Department.dept_id;
```

I/O | AI Agent — 225 ms

STDIN

Input for the program (Optional)

Output:

student_id	student_name	dept_name
101	Anu	CSE
102	Rahul	ECE

queries.sql

```sql
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(30) UNIQUE
);
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
SELECT Student.student_name,
       Department.dept_name
FROM Student
LEFT OUTER JOIN Department
ON Student.dept_id = Department.dept_id;
```

I/O | AI Agent — 290 ms

STDIN

Input for the program (Optional)

Output:

student_name	dept_name
Anu	CSE
Rahul	ECE
Kiran	NULL

queries.sql

```sql
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(30) UNIQUE
);
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
SELECT Student.student_name,
       Department.dept_name
FROM Student
RIGHT OUTER JOIN Department
ON Student.dept_id = Department.dept_id;
```

I/O | AI Agent — 292 ms

STDIN

Input for the program (Optional)

Output:

student_name	dept_name
Anu	CSE
Rahul	ECE
NULL	BCA
NULL	MECH

queries.sql

```sql

CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(30) UNIQUE
);
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
SELECT Student.student_name,
       Department.dept_name
FROM Student
LEFT JOIN Department
ON Student.dept_id = Department.dept_id
UNION
SELECT Student.student_name,
       Department.dept_name
FROM Student
RIGHT JOIN Department
ON Student.dept_id = Department.dept_id;
```

I/O | AI Agent | 330 ms

STDIN

Input for the program (Optional)

Output:

student_name	dept_name
NULL	BCA
NULL	MECH
Anu	CSE
Kiran	NULL
Rahul	ECE

queries.sql

```sql

CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(30) UNIQUE
);
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
SELECT Student.student_name,
       Department.dept_name
FROM Student
CROSS JOIN Department;
```

I/O | AI Agent | 264 ms

STDIN

Input for the program (Optional)

Output:

student_name	dept_name
Anu	CSE
Anu	ECE
Anu	BCA
Anu	MECH
Rahul	CSE
Rahul	ECE
Rahul	BCA
Rahul	MECH
Kiran	CSE
Kiran	ECE
Kiran	BCA
Kiran	MECH

Lab -8
Group by and Ordering

queries.sql

```sql
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(30) UNIQUE
);
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
SELECT dept_id, COUNT(*) AS total_students
FROM Student
GROUP BY dept_id;
```

I/O | AI Agent | 279 ms

STDIN

Input for the program (Optional)

Output:

```
dept_id     total_students
----------- --------------
       NULL              1
          1              1
          2              1
```

queries.sql

```sql
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(30) UNIQUE
);
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
SELECT dept_id, AVG(marks) AS average_marks
FROM Student
GROUP BY dept_id
HAVING AVG(marks) > 80;
```

I/O | AI Agent | 268 ms

STDIN

Input for the program (Optional)

Output:

```
dept_id     average_marks
----------- ---------------------------------------
          1                              85.000000
```

queries.sql

```sql
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(30) UNIQUE
);
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
SELECT student_name, marks
FROM Student
ORDER BY marks ASC;
```

I/O | AI Agent | 246 ms

STDIN

Input for the program (Optional)

Output:

```
student_name                                       marks
-------------------------------------------------- -------
Rahul                                                72.00
Kiran                                                75.00
Anu                                                  85.00
```

queries.sql

```sql

CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(30) UNIQUE
);
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
SELECT student_name, marks
FROM Student
ORDER BY marks DESC;
```

I/O | AI Agent | 277 ms

STDIN

Input for the program (Optional)

Output:

```
student_name                                       marks
-------------------------------------------------- -------
Anu                                                 85.00
Kiran                                               75.00
Rahul                                               72.00
```

queries.sql

```sql

CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(30) UNIQUE
);
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
SELECT dept_id,
       COUNT(*) AS total_students,
       AVG(marks) AS average_marks
FROM Student
GROUP BY dept_id
HAVING AVG(marks) > 70
ORDER BY average_marks DESC;
```

I/O | AI Agent

STDIN

Input for the program (Optional)

Output:

```
dept_id     total_students average_marks
----------- -------------- ----------------------------------------
          1              1                               85.000000
       NULL              1                               75.000000
          2              1                               72.000000
```

Lab -9
Procedures

queries.sql

```sql
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(30) UNIQUE
);
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
DELIMITER //
CREATE PROCEDURE GetAllStudents()
BEGIN
    SELECT * FROM Student;
END //
DELIMITER ;
CALL GetAllStudents();
```

I/O | AI Agent — 97 ms

STDIN

Input for the program (Optional)

Output:

```
+------------+--------------+---------+-------+------+
| student_id | student_name | dept_id | marks | age  |
+------------+--------------+---------+-------+------+
|        101 | Anu          |       1 | 85.00 |   20 |
|        102 | Rahul        |       2 | 72.00 |   21 |
|        106 | Kiran        |    NULL | 75.00 |   22 |
+------------+--------------+---------+-------+------+
```

queries.sql

```sql
CREATE TABLE Student (
    student_name VARCHAR(50),
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
DELIMITER //
CREATE PROCEDURE InsertStudent(
    IN p_id INT,
    IN p_name VARCHAR(50),
    IN p_dept INT,
    IN p_marks DECIMAL(5,2),
    IN p_age INT
)
BEGIN
    INSERT INTO Student
        (student_id, student_name, dept_id, marks, age)
    VALUES
        (p_id, p_name, p_dept, p_marks, p_age);
END //
DELIMITER ;
CALL InsertStudent(107, 'Neha', 1, 90, 20);
SELECT * FROM Student;
```

I/O | AI Agent — 75 ms

STDIN

Input for the program (Optional)

Output:

```
+------------+--------------+---------+-------+------+
| student_id | student_name | dept_id | marks | age  |
+------------+--------------+---------+-------+------+
|        101 | Anu          |       1 | 85.00 |   20 |
|        102 | Rahul        |       2 | 72.00 |   21 |
|        106 | Kiran        |    NULL | 75.00 |   22 |
|        107 | Neha         |       1 | 90.00 |   20 |
+------------+--------------+---------+-------+------+
```

queries.sql

```sql
CREATE TABLE Department (
    dept_name VARCHAR(30) UNIQUE
);
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
DELIMITER //
CREATE PROCEDURE UpdateMarks(
    IN p_id INT,
    IN p_marks DECIMAL(5,2)
)
BEGIN
    UPDATE Student
    SET marks = p_marks
    WHERE student_id = p_id;
END //
DELIMITER ;
CALL UpdateMarks(107, 95);
SELECT * FROM Student;
```

I/O | AI Agent | 89 ms

STDIN

Input for the program (Optional)

Output:

```
+------------+--------------+---------+------+------+
| student_id | student_name | dept_id | marks | age  |
+------------+--------------+---------+------+------+
|        101 | Anu          |       1 | 85.00 |   20 |
|        102 | Rahul        |       2 | 72.00 |   21 |
|        106 | Kiran        |    NULL | 75.00 |   22 |
+------------+--------------+---------+------+------+
```

queries.sql

```sql
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(30) UNIQUE
);
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
DELIMITER //
CREATE PROCEDURE DeleteStudent(
    IN p_id INT
)
BEGIN
    DELETE FROM Student
    WHERE student_id = p_id;
END //
DELIMITER ;
CALL DeleteStudent(107);
SELECT * FROM Student;
```

I/O | AI Agent | 87 ms

STDIN

Input for the program (Optional)

Output:

```
+------------+--------------+---------+-------+------+
| student_id | student_name | dept_id | marks | age  |
+------------+--------------+---------+-------+------+
|        101 | Anu          |       1 | 85.00 |   20 |
|        102 | Rahul        |       2 | 72.00 |   21 |
|        106 | Kiran        |    NULL | 75.00 |   22 |
+------------+--------------+---------+-------+------+
```

Lab -10
Triggers

queries.sql

```
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
CREATE TABLE Student_Audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT,
    action_type VARCHAR(20),
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
DELIMITER //
CREATE TRIGGER AfterInsert
AFTER INSERT ON Student
FOR EACH ROW
BEGIN
    INSERT INTO Student_Audit (student_id, action_type)
    VALUES (NEW.student_id, 'INSERT');
END //
DELIMITER ;
INSERT INTO Student VALUES (108, 'Asha', 1, 85, 20);
SELECT * FROM Student_Audit;
```

I/O | AI Agent | 99 ms

STDIN

Input for the program (Optional)

Output:

```
+----------+------------+-------------+---------------------+
| audit_id | student_id | action_type | action_time         |
+----------+------------+-------------+---------------------+
|        1 |        108 | INSERT      | 2026-10-11 14:11:18 |
+----------+------------+-------------+---------------------+
```

```
CREATE TABLE Student (
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
CREATE TABLE Student_Audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT,
    action_type VARCHAR(20),
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
DELIMITER //
CREATE TRIGGER AfterUpdate
AFTER UPDATE ON Student
FOR EACH ROW
BEGIN
    INSERT INTO Student_Audit
        (student_id, action_type)
    VALUES
        (NEW.student_id, 'UPDATE');
END //
DELIMITER ;
UPDATE Student
SET marks = 90
WHERE student_id = 101;
SELECT * FROM Student_Audit;
```

STDIN

Input for the program (Optional)

Output:

```
+----------+------------+-------------+---------------------+
| audit_id | student_id | action_type | action_time         |
+----------+------------+-------------+---------------------+
|        1 |        101 | UPDATE      | 2026-10-11 14:14:09 |
+----------+------------+-------------+---------------------+
```

queries.sql

```
CREATE TABLE Student (
    dept_id INT,
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
CREATE TABLE Student_Audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT,
    action_type VARCHAR(20),
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
DELIMITER //
CREATE TRIGGER BeforeDelete
BEFORE DELETE ON Student
FOR EACH ROW
BEGIN
    IF OLD.student_id = 101 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete student 101';
    END IF;
END //
DELIMITER ;
DELETE FROM Student
WHERE student_id = 101;
```

I/O | AI Agent | Error

STDIN

Input for the program (Optional)

Output:

```
ERROR 1644 (45000) at line 39: Cannot delete student 101
```

Debug this error

queries.sql

```
CREATE TABLE Student (
    marks DECIMAL(5,2),
    age INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
CREATE TABLE Student_Audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT,
    action_type VARCHAR(20),
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
INSERT INTO Department VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'BCA'),
(4, 'MECH');
INSERT INTO Student VALUES
(101, 'Anu', 1, 85, 20),
(102, 'Rahul', 2, 72, 21),
(106, 'Kiran', NULL, 75, 22);
DELIMITER //
CREATE TRIGGER AfterDelete
AFTER DELETE ON Student
FOR EACH ROW
BEGIN
    INSERT INTO Student_Audit
        (student_id, action_type)
    VALUES
        (OLD.student_id, 'DELETE');
END //
DELIMITER ;
DELETE FROM Student
WHERE student_id = 108;
SELECT * FROM Student_Audit;
```

I/O | AI Agent

STDIN

Input for the program (Optional)

Output:

```
Program did not output anything!
```